

9.3. Pipelining Internal Parameters

TMC4361 provides a target pipeline for sequencing subordinate targets in order to easily arrange a complex target structure.

9.3.1. Configuration and Activation of Target Pipeline

The different target values must be assigned to the $X_PIPE0...7$ register. If the target pipeline is enabled, a new assignment cycle is initiated as soon as an internal start signal is generated; moving the values, as described, simultaneously:

PROCESS DESCRIPTION:

- A new $XTARGET$ value is assigned that takes over the value of X_PIPE0 .
- Every X_PIPEn register takes over the value of its successor:
 $X_PIPEn = X_PIPEn+1$.

In order to activate the target pipeline, do as follows:

Action:

- Set $pipeline_en = b'0001$ ($START_CONF$ register 0x02).

Result:

The above mentioned process description is executed with every new internal start signal prompting.

Configuration of a cyclic Target Pipeline

It is also possible to reassign the value of $XTARGET$ to one (or more) of the pipeline registers $X_PIPE0...7$. Thereby, a cyclic target pipeline is created.

In order to enable a cyclic target pipeline, do as follows:

Action:

- Set $pipeline_en = b'0001$ ($START_CONF$ register 0x02).
- Set $XPIPE_REWRITE_REG$ in relation to the pipeline register where $XTARGET$ have to be written back (e.g. $XPIPE_REWRITE_REG = b'00010000$).

Result:

The above mentioned process description is executed with every new internal start signal prompting, and $XTARGET$ is written back to the selected X_PIPEx register (e.g. $XPIPE_REWRITE_REG = 0x10 \rightarrow XTARGET$ is written back to X_PIPE4).

The processes and actions described on the previous page, are depicted in the following figure. The assignment cycle that is initiated when an internal start signal occurs is depicted.

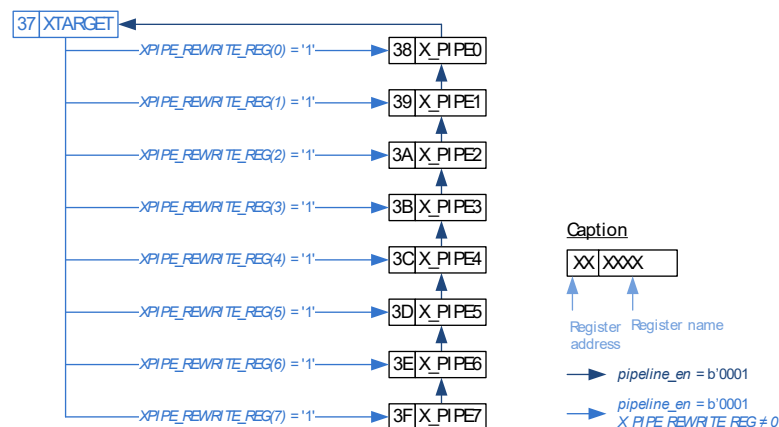


Figure 39: Target Pipeline with Configuration Options



9.3.2. Using the Pipeline for different internal Registers

The TMC4361 pipeline (registers 0x38...0x3F) can be configured so that it splits up into maximal four segments. These segments can be used to feed the following internal parameters:

- *XTARGET* register 0x37
- *POS_COMP* register 0x32
- *GEAR_RATIO* register 0x12
- *GENERAL_CONF* 0x00

Consequently, these definite parameter value changes can be of importance concerning a continuous ramp motion and/or for reduced overhead synchronizing of several motion controllers.

The *POS_COMP* value can be used to initiate a start signal generation during motion. Therefore, it can be useful to pipeline this parameter in order to avoid dependence on SPI transfer speed.

For instance, if the distance between two *POS_COMP* values is very close and the current velocity is high enough that it misses the second value before the SPI transfer is finished, it is advisable to change *POS_COMP* immediately after the start signal.

The same is true for the *GEAR_RATIO* parameter, which defines the step response on incoming step impulses. Some applications require very quick gear factor alteration of the slave controller. Note that when the start signal is prompted directly, an immediate change can be very useful instead of altering the parameter by an SPI transfer.

Likewise, it can (but must not) be essential to change general configuration parameters at a defined point in time. A suitable application is a clearly defined transfer from a direct external control (*sd_in_mode* = b'01) to an internal ramp (*sd_in_mode* = b'00) or vice versa because in this case the master/slave relationship is interchanged.

The following pipeline options are available, which can be adjusted accordingly:

Pipeline Activation Options	
<i>pipeline_en</i> (3:0)	Description
b'xxx1	Pipeline for <i>XTARGET</i> is enabled.
b'xx1x	Pipeline for <i>POS_COMP</i> is enabled.
b'x1xx	Pipeline for <i>GEAR_RATIO</i> is enabled.
b'1xxx	Pipeline for <i>GENERAL_CONF</i> is enabled.

Table 35: Pipeline Activation Options



9.3.3. Pipeline Mapping Overview

The *pipeline_en* parameter offers an open configuration for 16 different combinations of the pipeline segregation. As a result, the number of pipelines range from 0 to 4. This also has an impact on the pipeline depth. The possible options are as follows: eight stages, four stages, three stages and two stages.

In the “Pipeline Mapping” table below, the arrangement and depth of the pipeline is allocated according to the pipeline setup. The final register destination of pipeline registers are also depicted in order to illustrate from which pipeline registers (*X_PIPE0...7*) the final target registers (*XTARGET*, *POS_COMP*, *GEAR_RATIO*, *GENERAL_CONF*) are fed.

For example, if *POS_COMP* and *GEAR_RATIO* are chosen as parameters that are to be fed by the pipeline, two 4-stage pipelines are created. When an internal start signal is generated, *POS_COMP* assumes the value of *X_PIPE0*, whereas *X_PIPE4* feeds the *GEAR_RATIO* register.

But if *POS_COMP*, *GEAR_RATIO* and *XTARGET* are selected as parameter destinations, two 3-stage pipelines and one double-stage pipeline are created. When an internal start signal is generated, *XTARGET* assumes the value of *X_PIPE0*, *POS_COMP* assumes the value of *X_PIPE3*, whereas *X_PIPE6* feeds the *GEAR_RATIO* register.

Pipeline Mapping Table

More examples are described in detail on the following pages - explaining some of the possible configurations and referencing examples - listed in the table below.

Pipeline Mapping						
Ex.	<i>pipeline_en</i> (3:0)	Arrangement	Final transfer register for...			
			<i>GENERAL_CONF</i> → <i>pipeline_en</i> (3)	<i>GEAR_RATIO</i> → <i>pipeline_en</i> (2)	<i>POS_COMP</i> → <i>pipeline_en</i> (1)	<i>XTARGET</i> → <i>pipeline_en</i> (0)
-	<i>b'0000</i>	No Pipelining	-	-	-	-
-	<i>b'0001</i>	One 8-stage pipeline	-	-	-	<i>X_PIPE0</i>
A	<i>b'0010</i>		-	-	<i>X_PIPE0</i>	-
B	<i>b'0100</i>		-	<i>X_PIPE0</i>	-	-
-	<i>b'1000</i>		<i>X_PIPE0</i>	-	-	-
C	<i>b'0011</i>	Two 4-stage pipelines	-	-	<i>X_PIPE4</i>	<i>X_PIPE0</i>
-	<i>b'0101</i>		-	<i>X_PIPE4</i>	-	<i>X_PIPE0</i>
-	<i>b'1001</i>		<i>X_PIPE4</i>	-	-	<i>X_PIPE0</i>
-	<i>b'0110</i>		-	<i>X_PIPE4</i>	<i>X_PIPE0</i>	-
-	<i>b'1010</i>		<i>X_PIPE4</i>	-	<i>X_PIPE0</i>	-
D	<i>b'1100</i>		<i>X_PIPE4</i>	<i>X_PIPE0</i>	-	-
F	<i>b'0111</i>	Two 3-stage pipelines and one double-stage pipeline	-	<i>X_PIPE6</i>	<i>X_PIPE3</i>	<i>X_PIPE0</i>
-	<i>b'1011</i>		<i>X_PIPE6</i>	-	<i>X_PIPE3</i>	<i>X_PIPE0</i>
E	<i>b'1101</i>		<i>X_PIPE6</i>	<i>X_PIPE3</i>	-	<i>X_PIPE0</i>
-	<i>b'1110</i>		<i>X_PIPE6</i>	<i>X_PIPE3</i>	<i>X_PIPE0</i>	-
G/H	<i>b'1111</i>	Four double-stage pipelines	<i>X_PIPE6</i>	<i>X_PIPE4</i>	<i>X_PIPE2</i>	<i>X_PIPE0</i>

Table 36: Pipeline Mapping for different Pipeline Configurations



9.3.4. **Cyclic Pipelining**

For all of the above shown configuration examples, it is possible to write back the current values of the selected registers (*XTARGET*, *POS_COMP*, *GEAR_RATIO* and/or *GENERAL_CONF*) to any of the pipeline registers of their assigned pipeline in order to generate cyclic pipelines.

By selecting proper *XPIPE_REWRITE_REG*, the value that is written back to the pipeline register is selected automatically to fit the selected pipeline mapping.

9.3.5. **Pipeline Examples**

Below, several pipeline mapping examples with the corresponding configuration are shown.

Examples A+B:
Using one Pipeline

Example A: Cyclic pipeline for *POS_COMP*, which has eight pipeline stages.

Example B: Cyclic pipeline for *GEAR_RATIO*, which has six pipeline stages.

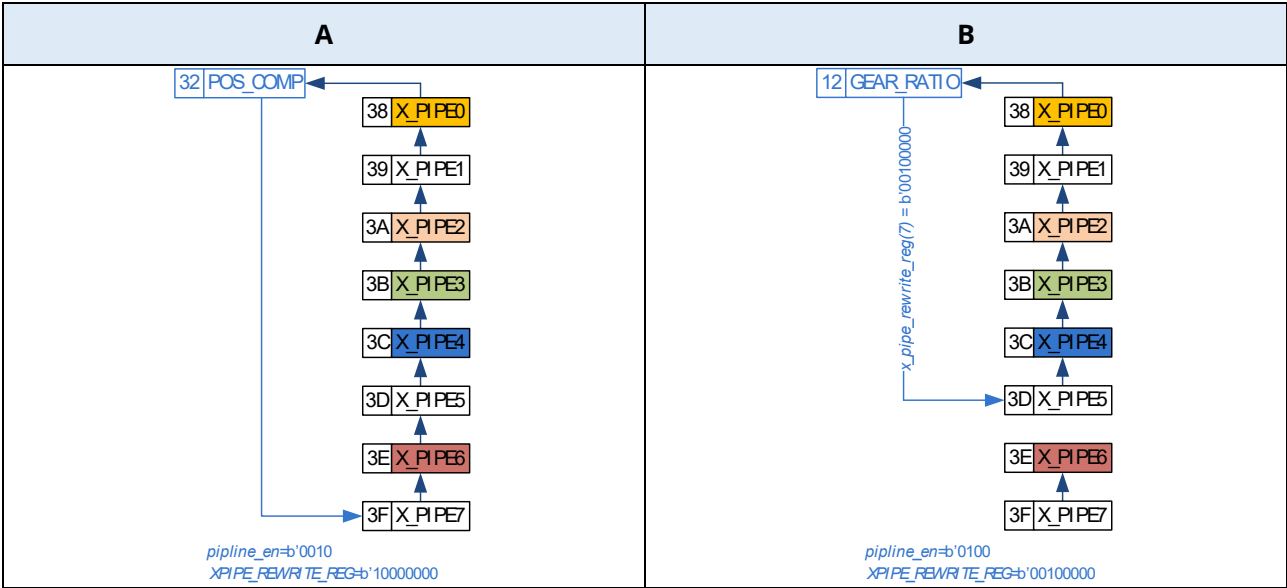


Figure 40: Pipeline Example A

Figure 41: Pipeline Example B

•→ Description is continued on next page.



Examples C+D: Using two Pipelines

Example C: Cyclic pipelines for *XTARGET* and *POS_COMP*, which have four pipeline stages each.

Example D: Cyclic pipelines for *GEAR_RATIO*, which has three pipeline stages and *GENERAL_CONF*, which has two pipeline stages.

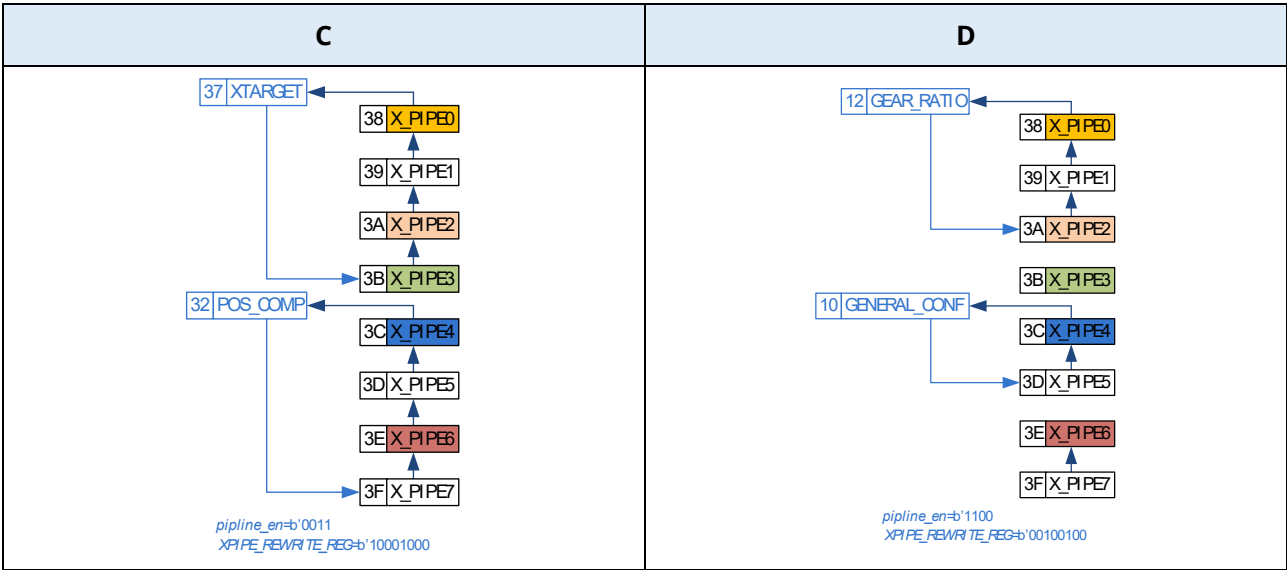


Figure 42: Pipeline Example C

Figure 43: Pipeline Example D

Examples E+F: Using three Pipelines

Example E: Cyclic pipelines for *XTARGET* and *GEAR_RATIO*, which have three pipeline stages each and *GENERAL_CONF*, which has two pipeline stages.

Example F: Two cyclic pipelines for *XTARGET* and *GEAR_RATIO*, which have two pipeline stages each and a noncyclic pipeline for *GEAR_RATIO*, which has three pipeline stages.

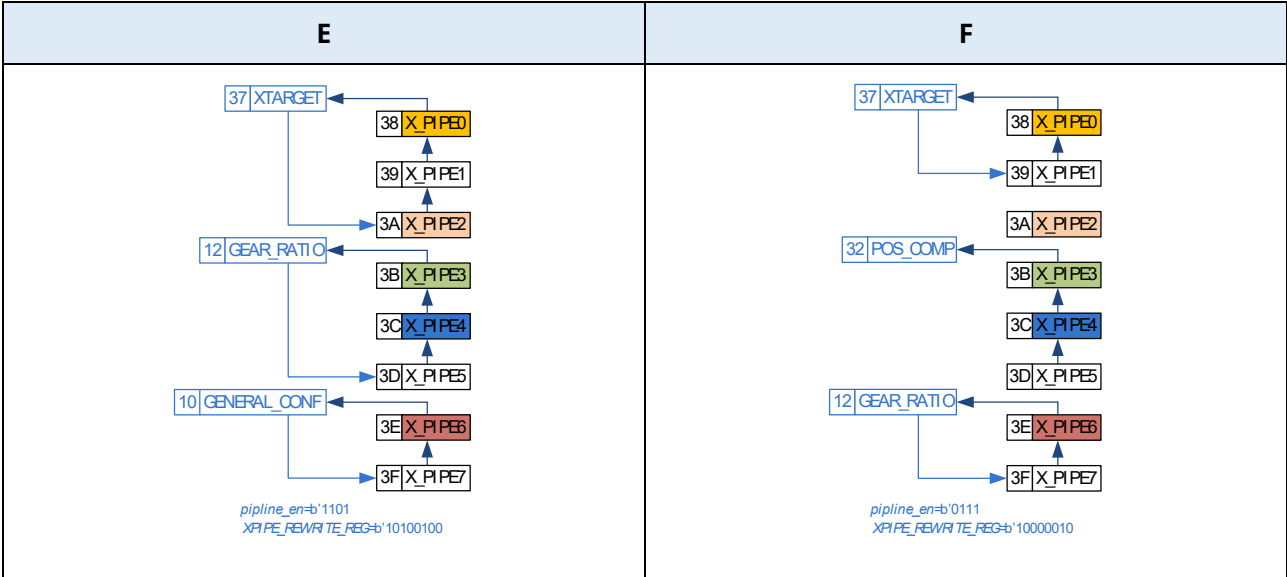


Figure 44: Pipeline Example E

Figure 45: Pipeline Example F

•→ Continued on next page.



Examples G+H:
Using four
Pipelines

Example G: Cyclic pipelines for *XTARGET*, *POS_COMP*, *GEAR_RATIO* and *GENERAL_CONF*, which have two pipeline stages each.

Example H: Four noncyclic pipelines for *XTARGET*, *POS_COMP*, *GEAR_RATIO* and *GENERAL_CONF*, which have two pipeline stages each.

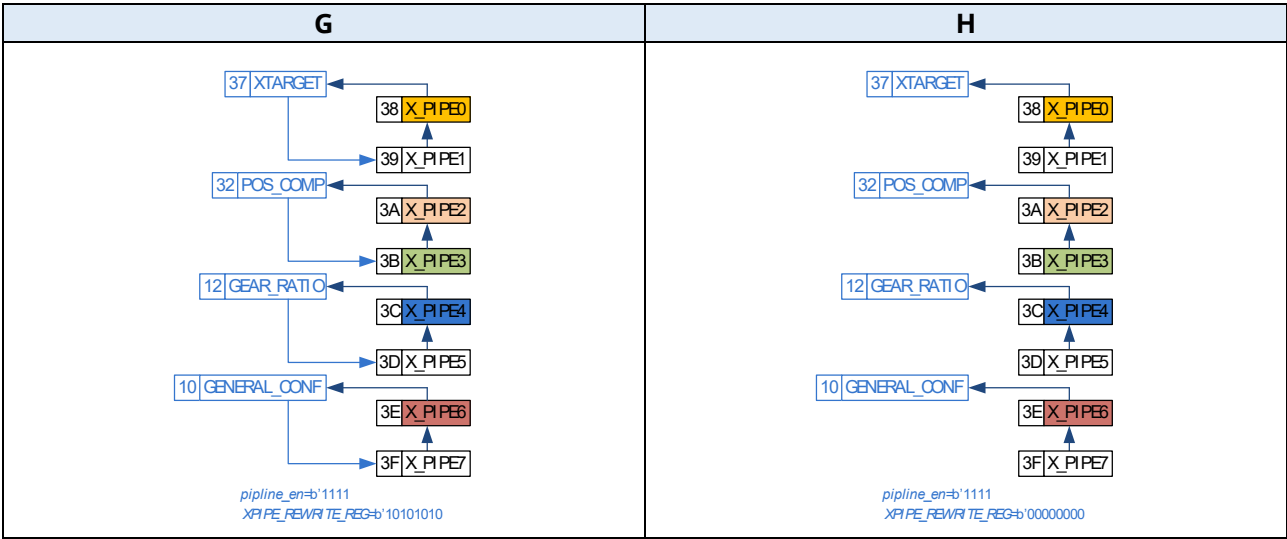


Figure 46: Pipeline Example G

Figure 47: Pipeline Example H



9.4. Masterless Synchronization of Several Motion Controllers via START Pin

START pin can also be assigned as tristate input in order to synchronize several microcontroller masterless.

Activation of the Tristate START Pin

In this case START is assigned as tristate. A busy state is enabled. During this busy state, START is set as output with a strongly driven inactive polarity. If the internal start signal is generated – after the internal start timer is expired – START pin is assigned as input. Additionally, a weak output signal is forwarded at START. During this phase, the active start polarity is emitted.

In case the signal at START input is set to active polarity, e.g. because all members of the signal line are ready, START output remains active (strong driving strength) for `START_OUT_ADD` clock cycles.

Then, busy state is active again until the next start signal occurs.

In order to activate tristate START pin, do as follows:

Action:

➤ Set `busy_en = 1` (`START_CONF` register 0x02).

Result:

The above mentioned process description is executed.

START Pin Connection

In case START pin is connected with START pins of other TMC4361 devices, it is recommend that a series resistor (e.g. 220 Ω) is connected between the devices to limit the short circuit current flowing that can flow during the configuration phase when different voltage levels at the START pins of the different devices can occur.

NOTE:

→ *Avoid that short circuits last too long.*



10. Serial Data Output

TMC4361 provides an SPI interface for initialization and configuration of the motor driver (in addition to the Step/Dir output) before and during motor motion. It is possible to control TMC stepper drivers during SPI motor drive.

SPI Interface Configuration

The SPI interface is used for the following tasks:

- TMC4361 integrates an adjustable cover register for configuration purposes in order to adjust TMC motor driver chips and third parties chips easily.
- The integrated microstep Sine Wave Lookup Table MSLUT generates two current values that represent sine and cosine values.
- These two current values can be transferred to a TMC motor driver chip at a time, in order to energize the motor coils. This occurs within each SPI datagram. A series of current values is transferred to move the motor. Values of the MSLUT are adjusted using velocity ramp dependent scale values that align the maximum amplitude current values to the requirements of certain velocity slopes.

Pin Names for SPI Motor Drive		
Pin Names	Type	Remarks
NSCSDRV_SDO	Output	Chip select output to motor driver, low active.
SCKDRV_NSDO	Output	Serial clock output to motor driver.
SDODRV_SCLK	InOut as Output	Serial data output to motor driver.
SDIDRV_NSCLK	Input	Serial data input from motor driver.
STDBY_CLK	Output	Clock output, standby output, or ChopSync clock output.

Table 37: Pin Names for SPI Motor Drive

Register Names for SPI Output Registers			
Register Name	Register Address		Remarks
GENERAL_CONF	0x00	RW	Affect switches: Bit14:13, bit19, bit20, bit28.
REFERENCE_CONF	0x01	RW	Affect switches: Bit26, bit27, bit30.
SPIOUT_CONF	0x04	RW	Configuration register for SPI output communication.
STEP_CONF	0x0A	RW	Microsteps per fullstep, fullsteps per revolution, and motor status bit event selection.
DAC_ADDR	0x1D	RW	SPI addresses/commands which are put in front of the DAC values: CoilA: DAC_ADDR(15:0), CoilB: DAC_ADDR(31:16)
SPI_SWITCH_VEL	0x1F	RW	Velocity at which automatic cover datagram are sent.
CHOPSYNC_DIV			Chopper clock divider (bit11:0).
FS_VEL	0x60	W	Velocity at which fullstep drive are enabled.
COVER_LOW	0x6C	W	Lower 32 bits of the cover register (µC to motor driver).
COVER_HIGH	0x6D	W	Upper 32 bits of the cover register (µC to motor driver).
COVER_DRV_LOW	0x6E	R	Lower 32 bits of the cover response register (motor driver to µC).
COVER_DRV_HIGH	0x6F	R	Upper 32 bits of the cover response register



Register Names for SPI Output Registers			
Register Name	Register Address		Remarks
			(motor driver to μ C).
<i>CURRENT_CONF</i>	0x05	RW	Current scaling configuration.
<i>SCALE_VALUES</i>	0x06	RW	Current scaling values.
<i>STDBY_DELAY</i>	0x15	RW	Delay time after standby mode is valid.
<i>FREEWHEEL_DELAY</i>	0x16	RW	Delay time after freewheeling is valid.
<i>VDRV_SCALE_LIMIT</i>	0x17	RW	Velocity setting for changing the drive scale value.
<i>UP_SCALE_DELAY</i>	0x18	RW	Increment delay to a higher scaling value; 24 bits.
<i>HOLD_SCALE_DELAY</i>	0x19	RW	Decrement delay to the hold scaling value; 24 bits.
<i>DRV_SCALE_DELAY</i>	0x1A	RW	Decrement delay to the drive scaling value.
<i>BOOST_TIME</i>	0x1B	RW	Delay time after ramp start when boost scaling is valid.
<i>SCALE_PARAM</i>	0x7C	R	Actual current scaling parameter; 8 bits.
<i>CURRENTA</i> <i>CURRENTB</i>	0x7A	R	Actual current values of the MSLUT: SIN (coil A) and SIN90_120 (coil B); 9 bits for each.
<i>CURRENTA_SPI</i> <i>CURRENTB_SPI</i>	0x7B	R	Actual scaled current values of the MSLUT: SIN (coil A) and SIN90_120 (coil B); 9 bits for each.
<i>MSLUT registers</i>	0x70...78	W	MSLUT values definitions.
<i>MSCNT</i>	0x79	R	Actual microstep position of the MSLUT.
<i>START_SIN</i> <i>START_SIN90_120</i> <i>DAC_OFFSET</i>	0x7E	RW	Sine start value of the MSLUT (bit 7:0). Cosine start value of the MSLUT (bit 23:16). Offset value for DAC output values (bit 31:24).

Table 38: Dedicated SPI Output Registers

10.1. Getting Started with TMC Motor Drivers

In this chapter information is provided about how to easily start up a connected TMC motor driver.

Setting up *SPIOUT_CONF* correctly

In order to start up a connected TMC motor stepper driver, proper setup of *SPIOUT_CONF* register 0x04 is important. TMC4361 offers presets for current transfer and automatic configuration routines if the correct TMC driver is selected. Status bits of TMC motor drivers are also transmitted to the status register of the motion controller.

TMC4361 provides a programmable lookup table (LUT) for storing the current wave. Per default, the tables are preprogrammed with a sine wave, which is a good starting point for most stepper motors.



10.2. Sine Wave Lookup Tables

TMC4361 provides a programmable lookup table (LUT) for storing the current wave. Reprogramming the table from its predefined values to a motor-specific wave allows improved motor-reliant microstepping, particularly when using low-cost motors.

SETTINGS ALERT



TMC4631-LA provides a default configuration of the internal microstep table MSLUT. In case internal MSLUT is used, proceed with section 10.9. (page 97) in order to configure a well-defined serial data connection to the stepper motor driver. The following explanations that are provided in this section only address engineers who use their own microstep table definition.

Programming Sine Wave Lookup Tables

The internal microstep wave table maps the microstep wave from 0° to 90° for 256 microsteps. It becomes automatically and symmetrically extended to 360° that consequently comprises 1024 microsteps. As a result, the microstep counter *MSCNT* ranges from 0 to 1023. Only a quarter of the wave is stored because this minimizes required memory and the amount of programmable data.

Therefore, only 256 bits (*ofs00* to *ofs255*) are required to store the quarter wave. These bits are mapped to eight 32-bit registers *MSLUT*[0] (register 0x70) to *MSLUT*[7] (register 0x77).

When reading out the table the 10-bit microstep counter *MSCNT* addresses the fully extended wave table.

Sine Wave Table Structure

The MSLUT is an incremental table. This means that a certain order and succession is predefined at every next step based on the value before, using up to four flexible programmable segments within the quarter wave. The microstep limits of the four segments are controlled by the position registers *X1*, *X2*, and *X3*.

Within these segments the next value of the MSLUT is calculated by adding the base wave inclination $W_x - 1$ (if *ofs*=0) or its successor W_x (if *ofs*=1). Because four segments are programmable, four base wave inclinations are available as basic increment value: 0, 1, 2, or 3. Thereby, even a negative wave inclination can be realized. This is shown in the next Figure where the values in last quarter segments are decreased or remain constant with every step towards *MSCNT*= 255.

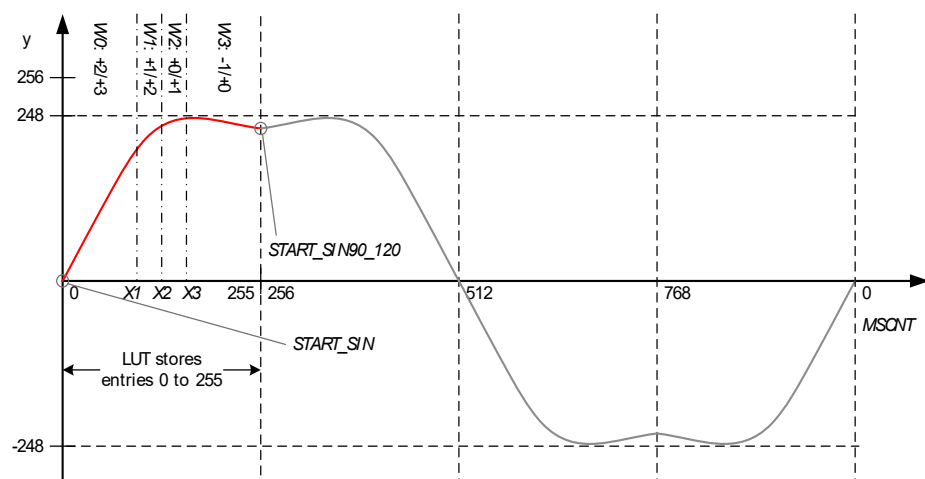


Figure 48: LUT Programming Example



10.2.1. Actual Current Values Output

Actual Current Calculations

When the microstep sequencer advances within the microstep table (MSLUT), it calculates the actual current values for the motor coils with each microstep, and stores them to the register 0x7A, which comprises the values of both waves *CURRENTA* and *CURRENTB*. However, the incremental coding requires an absolute initialization – especially when the microstep table becomes modified. Therefore, *CURRENTA* and *CURRENTB* become re-initialized with the start values whenever *MSCNT* passes zero.

Characteristics of a 2-Phase Stepper Motor Microstep Table

As mentioned above, the MSLUT can be adapted to the motor requirements. In order to understand the nature of incremental coding of the microstep table, the characteristics of the microstep wave must be understood, as described in the list below:

Characteristics of a 2-phase motor microstep table:

- In principle, it is a reverse characteristic of the motor pole behavior.
- It is a polished wave to provide a smooth motor behavior. There are no jumps within the wave.
- The phase shift between both phases is exactly 90°, because this is the optimum angle of the poles inside the motor.
- The zero transition is at 0°. The curve is symmetrical within each quadrant (like a sine wave).
- The slope of the wave is normally positive, but due to torque variations it can also be (slightly) negative.
- But it must not be strictly monotonic as shown in the figure above.

Considering these facts, it becomes clear that the wave table can be compressed. The incremental coding applied to TMC4361 uses a format that reduces the required information - *per entry of the 8-bit by a 256-entry wave table* - to slightly more than a single bit.



10.2.2. How to Program the Internal MSLUT

Principle of Incremental Encoding

The principle of **incremental encoding** only stores the difference between the actual and the next table entry. In order to attain an absolute start value, the first entry is directly stored in *START_SIN*. Also, for ease-of-use, the first entry of the shifted table for the second motor phase is stored in *START_SIN_90_120*.

Based on these start values, every next table entry is calculated by adding an increment INC to the former value. This increment is the base wave inclination value W_x whenever its corresponding *ofs* bit is 1 or $W_x - 1$ if *ofs* = 0:

$$INC = W_x + (ofs - 1).$$

The base wave inclination can be set to four different values (0, 1, 2, 3), because it consists of two bits.

Because the wave inclination does not change dramatically, TMC4361 provides four wave inclination segments with the base wave inclinations (W_0 , W_1 , W_2 , and W_3) and the segment borders (0, X_1 , X_2 , X_3 , and 255), as shown in the left quarter of the MSLUT diagram in *Figure 48*, page 90.

Wave Inclination Characteristics		
Wave Inclination Segment	Base Wave Inclination	Segment Ranges
0	W_0	0 ... X_1
1	W_1	X_1 ... X_2
2	W_2	X_2 ... X_3
3	W_3	X_3 ... 255

Table 39: Wave Inclination Characteristics of Internal MSLUT



10.2.3. Setup of MSLUT Segments

Base Wave Inclination and Border Values

All base wave inclination values (each consists of two bits) as well as the border values (each consists of eight bit) between the segments are adjustable. They are assigned by *MSLUTSEL* register 0x78.

In order to change the base wave inclination values and the segment borders, do as follows:

Action:

- Define the segment borders X1, X2, and X3 and the base wave inclination values W0...W3 according to the requirements
- Set register *MSLUTSEL*(31:24) = X3.
- Set register *MSLUTSEL*(23:16) = X2.
- Set register *MSLUTSEL*(15:8) = X1.
- Set register *MSLUTSEL*(7:6) = W3.
- Set register *MSLUTSEL*(5:4) = W2.
- Set register *MSLUTSEL*(3:2) = W1.
- Set register *MSLUTSEL*(1:0) = W0.

Result:

The segments and the base wave inclination values of the internal MSLUT are changed.

NOTE:

→ *It is not mandatory to define four segments. For instance, if only two segments are required, set X2 and X3 to 255. Then, W0 is valid for segment 0 between MSCNT = 0 and MSCNT = X1, and W1 is valid between MSCNT = X1 and MSCNT = 255 (segment 1).*

In order to change the ofs bits, do as follows:

Action:

- Set *MSLUT*[0] register 0x70 = ofs31...ofs00.
- Set *MSLUT*[1] register 0x71 = ofs63...ofs32.
- Set *MSLUT*[2] register 0x72 = ofs95...ofs64.
- Set *MSLUT*[3] register 0x73 = ofs127...ofs96.
- Set *MSLUT*[4] register 0x74 = ofs159...ofs128.
- Set *MSLUT*[5] register 0x75 = ofs191...ofs160.
- Set *MSLUT*[6] register 0x76 = ofs223...ofs192.
- Set *MSLUT*[7] register 0x77 = ofs255...ofs224.

Result:

The ofs bits of the internal MSLUT are changed.

AREAS OF SPECIAL CONCERN



When modifying the wave:

Special care has to be applied in order to ensure a smooth and symmetrical zero transition whenever the quarter wave becomes expanded to a full wave.

Zero Crossing

When adjusting the range:

The maximum resulting swing of the wave should be adjusted to a range of -248 to 248, in order to achieve the best possible resolution while at the same time leaving headroom for a hysteresis based chopper to add an offset.



10.2.4. Current Waves Start Values

Starting Current Values of MSLUT Configuration

As both waves are shifted by 90° for two-phase stepper motors, the sine wave starts at 0° when $MSCNT = 0$. By comparison, the cosine wave begins at 90° when $MSCNT = 256$. At this starting points the current values are $CURRENTA = 0$ for the sine wave and $CURRENTB = 247$ for the cosine wave.

In contrast to the starting microstep positions that are fixed, these starting current values can be redefined if the default start values do not fit for the actual MSLUT.

In order to change the starting current values of the MSLUT, do as follows:

Action:

- Define the start values $START_SIN$ and $START_SIN90_120$ according to the requirements.
- Set register 0x7E (7:0) = $START_SIN$.
- Set register 0x7E (23:16) = $START_SIN90_120$.

Result:

The starting values for both waves are adapted to MSLUT.

10.2.5. Default MSLUT

Base Wave Inclinations

The default sine wave table in TMC drivers uses one segment with a base inclination of 2 and one segment with a base inclination of 1 (see default value of the $MSLUTSEL$ register 0x78 = 0xFFFF8056).

The segment border X1 is located at $MSCNT = 128$. The base wave inclinations are $W0 = b'10 (=2)$ and $W1 = b'01 (=1)$.

As a result, between $MSCNT = 0$ and 128, the increment value INC is either 1 (if $ofs = 0$) or 2 (if $ofs = 1$).

And between $MSCNT = 128$ and 255, the increment value INC is either 0 (if $ofs = 0$) or 1 (if $ofs = 1$).

This reflects the stronger rise in the first segment of the MSLUT in contrast to the second segment. The maximum value is

$$START_SIN90_120 = 247.$$



10.2.6. Explanatory Notes for Base Wave Inclinations

Definition of Segments 0,1,2,3

In the following example four segments are defined.

Each segment has a different base wave inclination to illustrate each possible entry:

Segment 0: $W_0 = 3$ which means that the increment value is +2 or +3.

Segment 1: $W_0 = 2$ which means that the increment value is +1 or +2.

Segment 2: $W_0 = 1$ which means that the increment value is 0 or +1.

Segment 3: $W_0 = 0$ which means that the increment value is -1 or 0.

- i In addition to the MSLUT curve (black line), which is defined by the given *ofs* bits, all four segments show upper limits (red line); in case all *ofs* bits in the particular segments are set to 1.
- i The green line shows the lower limit in case all *ofs* bits in the particular segments are set to 0.

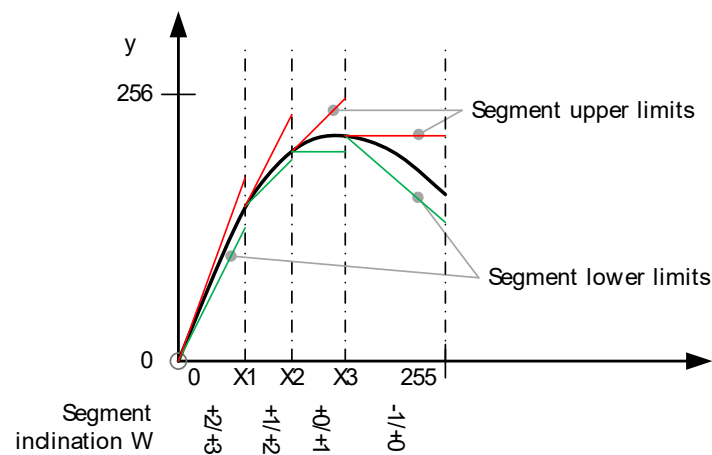


Figure 49: MSLUT Curve with all possible Base Wave Inclinations (highest Inclination first)

Standard Sine Wave Setup Considerations prior to SETUP of MSLUT

In order to set up a standard sine wave table for the MSLUT, the following considerations have to be taken into account:

PRECONSIDERATIONS:

- The microstep table for the standard sine wave begins with eight entries (0 to 7) {0, 1, 3, 4, 6, 7, 9, 10 ...} etc.
- The maximum difference between two values in this section is +2, whereas the minimum difference is +1.
- While advancing according to the table, the very first time the difference between two MSLUT values is lower than +1 is between position 153 and position 154. Both entries are identical.
- The start value is 0 for the sine wave.
- The calculated value for position 256 (i.e. start of cosine wave) is 247.

•→ Description is continued on next page.



Standard Sine Wave Setup

In order to set up the standard sine wave table, proceed as follows:

Action:

- Set a starting value $START_SIN = 0$ matching sine wave entry 0.
- Set a base wave inclination range of $W0 = b'10 = 2$ to skip between +1 / +2, valid from 0 to X1.
- Calculate the differences between every entry: {+1, +2, +1, +2, +1, +2, +1,...}.
- Set the microstep table entries $ofsXX$ to 0 for the lower value (+1); 1 for the higher value (+2). Thus, the first seven microstep table entries $ofs00$ to $ofs06$ are: {0, 1, 0, 1, 0, 1, 0 ...}
- The base wave inclination must be lowered at position 153, at very latest. Use the next base wave inclination range 1 with $W1 = b'01 = 1$ to skip between +0 and +1.
- Set $X1 = 153$ in order to switch to the next inclination range. From here on, an offset $ofsXX$ of 0 means add nothing; 1 means add +1.
- Set $START_SIN90_120 = 247$, which is equal to the value at position 256.
- Only two of four wave segments with different base wave inclinations are used. The remaining wave inclination ranges $W2$ and $W3$ should be set to the same value as $W1$; and $X2$ and $X3$ can be set to 255. Thereby, only two wave inclination segments are effective.

Result:

A standard sine wave is defined as MSLUT. The following table shows an extract of this curve.

Overview of the Microstep Behavior Example													
Microstep number	0	1	2	3	4	5	6	7	...	153	154	...	255
Desired table entry	0	1	3	4	6	7	9	10	...	200	200	...	247
Difference to next entry	1	2	1	2	1	2	1	...	...	0	...	...	0
Required segment inclination	+2	+2	+2	+2	+2	+2	+2	...	...	+1	...	...	+1
Ofs bit entry	0	1	0	1	0	1	0	...	...	0	...	...	0

Table 40: Overview of the Microstep Behavior Example



10.3. SPI Output Interface Configuration Parameters

TMC4361 provides an SPI output interface. In the next section, the configuration of the interface parameters is explained in detail.

10.3.1. Enabling SPI Output Communication

In order to enable SPI output communication, do as follows:

Action:

- Set `serial_enc_out_enable` = 0 (bit24 of `GENERAL_CONF` register 0x00).

Result:

SPI output is enabled.

- i SPI out is the default preconfigured setting.

Pins dedicated to SPI Output Communication

The table below lists the pins that are dedicated to SPI output communication:

SPI Output Communication Pins	
Pin	Description
NSCSDRV_SDO	Low active chip select signal.
SCKDRV_NS DOSPI	SPI output clock.
SDODRV_SCLK	MOSI – Output pin to transfer the datagram to the motor driver.
SDIDRV_NSCLK	MISO – Input pin which receives the response from the motor driver. The response is sampled during the data transfer to the motor driver.

Table 41: SPI Output Communication Pins



10.3.2. Setup of SPI Output Timing Configuration

Because TMC4361 represents the master of SPI communication to the motor driver – which is the slave – it is mandatory to set up the timing configuration for the SPI output. TMC4361 provides an SPI clock, which is generated at the SCKDRV_NSDO output pin.

In order to configure the timing of the SPI clock, set up *SPIOUT_CONF* register 0x04 as follows:

Action:

- Set the number of internal clock cycles the serial clock should stay low at $SPI_OUT_LOW_TIME = SPIOUT_CONF(23:20)$.
- Set the number of internal clock cycles the serial clock should stay high at $SPI_OUT_HIGH_TIME = SPIOUT_CONF(27:24)$.
- Also, an $SPI_OUT_BLOCK_TIME = SPIOUT_CONF(31:28)$ can be set for a minimum time period during which no new datagram is sent after the last SPI output datagram.

Result:

SPI output communication scheme is set. During the inactive phase between to SPI datagrams - which is at least $SPI_OUT_BLOCK_TIME$ clock cycles long - the SCKDRV_NSDO and NSCSDRV_SDO pins remain at high output voltage level. The timing of the SPI output communication is illustrated in the next Figure 50 (page 98).

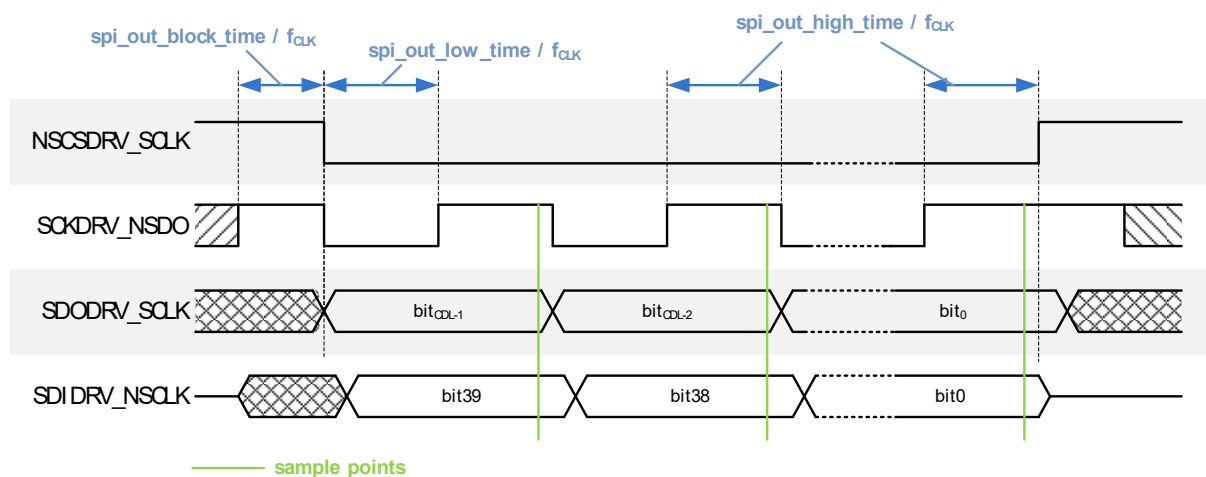


Figure 50: SPI Output Datagram Timing

Minimum and Maximum Time Period

The minimum time period for all three parameters is $2/f_{CLK}$. If an SPI output parameter is set to 0, it is altered to 2 clock cycles internally. A maximum time period of $15/f_{CLK}$ can be set for all three parameters.

Thus, SPI clock frequency f_{SPI_CLK} covers the following range:

$$f_{CLK} / 30 \leq f_{SPI_CLK} \leq f_{CLK} / 2.$$



10.3.3. Current Diagrams

Process Description

Basically, SPI output communication serves as automatic current datagram transfer to the connected motor driver. TMC4361 uses the internal microstep lookup table (MSLUT) in order to provide actual current motor driver data.

- With every step that is initialized by the ramp generator the *MSCNT* value is increased or decreased, dependent on ramp direction.
- The *MSCNT* register 0x79 (readable value) contains the current microstep position of the sine value.
- Accordingly, the current values *CURRENTA* (0x7A) and *CURRENTB* (0x7B) are altered.
- In case the output configuration of TMC4361 allows for automatic current transfer an updated current value leads to a new datagram transfer.
- Thereby, the motor driver always receives the latest data. The length for current datagrams can be set automatically and TMC4361 converts new values into the selected datagram format, usually divided in amplitude and polarity bit for TMC motor drivers.

10.3.4. Change of Microstep Resolution

By altering the microstep resolution from 256 (*MSTEP_PER_FS* = b'0000) to a lower value, an internal step results in more than one MSLUT step.

For instance, if the microstep resolution is set to 64 (*MSTEP_PER_FS* = b'0010), *MSCNT* is either increased or decreased by 4 per each internal step. Accordingly, the passage through the MSLUT skips three current values per each internal step to match the new microstep resolution.

10.3.5. Cover Datagrams Communication between µC and Driver

In addition to automatic current datagram transfer, the microcontroller can communicate directly with the motor driver through TMC4361 by using cover datagrams. This communication channel can be useful for configuration purposes because no additional SPI communication channel between microcontroller and motor driver is necessary.

Up to 64 bits can be assigned for one cover datagram. This 64-bit SPI cover register is separated into two 32-bit registers - *COVER_HIGH* register 0x6D and *COVER_LOW* register 0x6C. The *COVER_HIGH* register is only required if more than 32 bits must be sent once.

How to Define Cover Datagram Length

How many bits are sent within one cover datagram is defined by the cover datagram length *COVER_DATA_LENGTH*.

In order to define the cover datagram length, do as follows:

Action:

- Set the number of cover datagram bits at
COVER_DATA_LENGTH = *SPIOUT_CONF* (19:13).

Result:

The cover datagram length is set to *COVER_DATA_LENGTH* bits. If this parameter is set higher than 64, the cover register data length is still maximum 64 bits.

- i For TMC motor drivers it is possible to set *COVER_DATA_LENGTH* = 0. In this case, the cover data length is selected automatically, dependent on the chosen motor driver. More details are provided on the subsequent pages.



10.3.6. Sending Cover Datagrams

The LSB (last significant bit) of the whole cover datagram register is located at `COVER_LOW(0)`. As long as `COVER_DATA_LENGTH < 33`, only `COVER_LOW` or parts of this register are required for cover data transfer.

If more than 32 bits are necessary, the complete `COVER_LOW` and (parts of) the `COVER_HIGH` register are required for SPI cover data transfer.

NOTE:

→ Every SPI communication starts with the most significant bit (MSB).

OPTION 1: COVER DATA LENGTH < 33 BITS

In order to send a cover datagram - that is smaller than 33 bits - do as follows:

Action:

➤ Set `COVER_LOW (COVER_DATA_LENGTH-1:0)` register `0x6C = cover_data`.

Result:

After a valid register request to `COVER_LOW`, SPI output is sent out `COVER_DATA_LENGTH` bits of `COVER_LOW` register.

Cover Datagrams with 32 Bits

OPTION 2: COVER DATA LENGTH > 32 BITS

In order to send a cover datagram - that consists of more than 32 bits - do as follows:

Action:

➤ Split cover data into two segments:

➤ `cover_data_low = cover_data(31:0)`.

➤ `cover_data_high = cover_data >> 32`.

➤ `cover_data_high = cover_data(31:0)`.

➤ Set `COVER_HIGH(COVER_DATA_LENGTH-32:0)` register `0x6D=cover_data_high`.

➤ Set `COVER_LOW` register `0x6C = cover_data_low`.

Result:

After a valid register request to `COVER_LOW`, SPI output is sent out `COVER_DATA_LENGTH` bits that comprises register values of `COVER_HIGH` and `COVER_LOW`.

The cover register and the datagram structure are illustrated in the figure below:

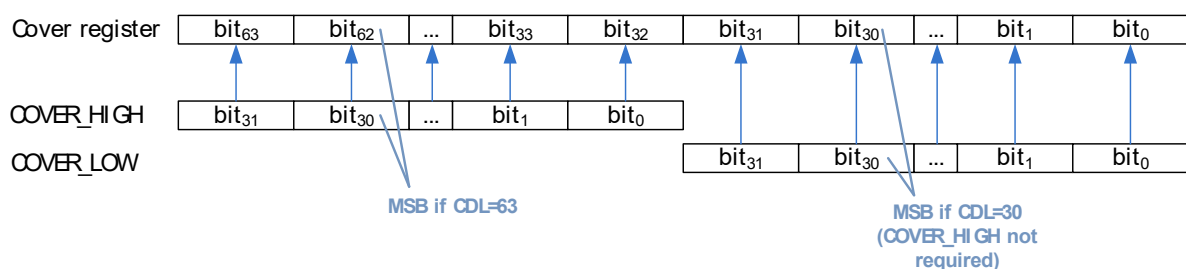


Figure 51: Cover Data Register Composition (CDL – `COVER_DATA_LENGTH`)

•→ Continued on next page.



Receiving Responses to Cover Datagrams

Because the transfer of a cover datagram is usually accompanied by a data transfer from the motor driver, the response is stored in registers; and is thus available for the microcontroller. `COVER_DRV_HIGH` register 0x6F and `COVER_DRV_LOW` register 0x6E form this cover response register that can also comprise up to 64 bits. Similar to `COVER_LOW` and `COVER_HIGH`, the motor driver response is divided in the registers `COVER_DRV_LOW` and `COVER_DRV_HIGH`. The composition of the response cover register and also the positioning of the MSB follow the same structure.

COVER_DONE Event

At the end of a successful data transmission, the event `COVER_DONE` becomes set. This indicates that the cover register data is sent to the motor driver and that the received response is stored in the `COVER_DRV_HIGH` register 0x6F and `COVER_DRV_LOW` register 0x6E.

10.3.7. Configuring Automatic Generation of Cover Datagrams

In certain setups, it can be useful to automatically send ramp velocity-dependent cover datagrams, e.g. to change chopper settings during motion.

NOTE:

→ *This feature is only available if the cover datagram length does not exceed 32 bits.*

In order to activate ramp velocity-dependent automatic cover data transfer, do as follows:

Action:

- Define the trigger velocity whenever an automatic cover datagram transfer is initiated.
- Set `SPI_SWITCH_VEL` register 0x1D to this absolute velocity [pps].
- Set `COVER_LOW` register 0x6C to the cover_data, which is valid for lower velocity values.
- Set `COVER_HIGH` register 0x6D to the cover_data, which is valid for higher velocity values.
- Set `automatic_cover` = 1 (`REFERENCE_CONF` register 0x01).

Result:

Whenever the absolute internal ramp velocity `|VACTUAL|` passes the `SPI_SWITCH_VEL` value, the particular cover data is sent to the motor driver, `COVER_LOW` is sent in case `|VACTUAL| < SPI_SWITCH_VEL`, `COVER_HIGH` is sent in case `|VACTUAL| ≥ SPI_SWITCH_VEL`.



10.4. Overview: TMC Motor Driver Connections

As mentioned before, TMC4361 is able to set the cover register length automatically in case a TMC motor driver is connected. Also, several additional automatic features for the SPI communication are available by selecting TMC motor drivers.

10.4.1. TMC Stepper Motor Driver Settings

Available SPI and Step/Dir™ Communication Schemes for TMC Motors

The SPI and Step/Dir communication schemes are available for the following product lines that are explained in greater detail further below:

- TMC236, TMC239
- TMC246, TMC248, TMC249
- TMC260, TMC261, TMC262, TMC2660
- TMC389
- TMC2130

How to enable SPI Output Settings for TMC Stepper Motor Drivers

In order to enable an operating SPI output setting for a connected TMC stepper motor driver, proceed as follows:

Action:

- Set *SPI_OUT_LOW_TIME*, *SPI_OUT_HIGH_TIME*, and *SPI_OUT_BLOCK_TIME* according to the TMC motor driver specification, as explained before.
- Set *COVER_DATA_LENGTH* = 0 (bit19:13 of *SPIOUT_CONF* register 0x04).
- Set *spi_output_format* = *SPI_OUT_CONF* (3:0) according to the connected SPI motor driver as seen below in the table below.

Result:

The communication scheme is now prepared for the connected TMC motor driver with all available features.

TMC Stepper Motor Driver Options				
TMC Motor Driver	<i>spi_output_format</i> = <i>SPI_OUT_CONF</i> (3:0)	Cover Register Datagram Length <i>COVER_DATA_LENGTH</i> =0	Automatic Current Datagram Transfer	Cover Register Datagram Transfer
SPI output off	b'0000	0	--	--
TMC23x	b'1000	12	✓	✓
TMC24x	b'1001	12	✓	✓
TMC26x/389	b'1010	20	✓	✓
	b'1011	20	S/D output	
TMC2130	b'1101	40	✓	✓
	b'1100	40	S/D output	

Table 42: TMC Stepper Motor Driver Options



10.4.2. TMC Motor Driver Response Datagram and Status Bits

When a TMC motor driver receives a current datagram or a cover datagram that is transmitted via SPI output of TMC4361, status data is sent back to the TMC4361 controller immediately. The response is stored in the *COVER_DRV_LOW* 0x6E and *COVER_DRV_HIGH* 0x6F registers, just like all other cover requests.

The type and sequence of the status bits that are sent back are dependent on the selected motor driver. A detailed list for every motor driver is presented in the next sections, in which the motor driver communication specifics for every driver family are explained separately.

The mapping of the available status bits to the TMC4361 *STATUS* register is similar for each and every TMC stepper motor driver. The last eight bits – *STATUS* (31:24) – are equal to the transferred motor status bits. A detailed overview is given in the register chapter [17.14.](#) (page [192](#)).

10.4.3. Events and Interrupts based on Motor Driver Status Bits

TMC4361 also provides one event at **EVENTS (30)** that is connected with the motor driver status bits. Here, any of the motor driver status bits can function as the base for this event.

In order to activate a motor driver status bit for the motor event *EVENTS* (30), do as follows:

Action:

- Selected one or more of the motor driver status for the motor event by assigning *MSTATUS_SELECTION* = *STEP_CONF* (23:16) register 0x0A accordingly.

Result:

In case one of the selected motor status bits is activated (Wired-Or), the motor event switch *EVENTS* (30) generates an event.

In order to generate an interrupt for this motor event, configure the INTR output accordingly, as explained in section [5.3.](#) (page [26](#)).



10.4.4. Stall Detection and Stop-on-Stall

stallGuard and stallGuard2 Functionality

TMC stepper motor driver chips with stallGuard and stallGuard2 can detect stall and overload conditions based on the motor's back-EMF without the need of a position sensor. The stall detection status is returned via SPI.

For more information, refer to the AppNote "Parameterization of stallGuard2 & coolStep" that is available online at www.trinamic.com.

Representation of Motor Stall Status

Except for TMC23x and TMC24x, which forward three load detection bits, the motor stall status is represented by one status bit. TMC4361 is able to stop the internal ramp as soon as a stall is recognized. Because stall bit activation can occur unwanted during motion with a low velocity, it is also possible to set up a velocity threshold for the Stop-on-Stall behavior.

Internal Velocity Ramp Stop-on-Stall Activation

In order to activate a Stop-on-Stall for the internal velocity ramp, do as follows:

Action:

- Set *VSTALL_LIMIT* register 0x67 [pps] according to minimum absolute velocity value for a correct stall recognition.
- Set *stop_on_stall* = 1 (bit26 of *REFERENCE_CONF* register 0x01).
- Set *drive_after_stall* = 0 (bit27 of *REFERENCE_CONF* register 0x01).

Result:

The internal ramp velocity is set immediately to 0 whenever a stall is detected and the following is true: $|V_{ACTUAL}| > V_{STALL_LIMIT}$.

Then, the *STOP_ON_STALL* event is also generated.

- i The status bit stallGuard that is directly mapped from the motor stepper driver, which is listed in *STATUS* (24). This flag is always activated as soon as the motor driver generates the stall guard status bit.
- i The *ACTIVE_STALL* status bit = *STATUS* (11) is activated as soon as a stall is detected and $|V_{ACTUAL}| > V_{STALL_LIMIT}$.

Internal Velocity Ramp Activation after Stop-on-Stall

In order to activate the internal velocity ramp AFTER a Stop-on-Stall, do as follows:

Action:

- Read out the *EVENTS* register 0x0E to unlock the event *STOP_ON_STALL*.
- Set *drive_after_stall* = 1 (bit27 of *REFERENCE_CONF* register 0x01).

Result:

The internal ramp velocity is no longer blocked by the Stop-on-Stall event.

- i In order to activate the Stop-on-Stall behavior again, reset *drive_after_stall* again manually to 0.



10.5. TMC23x, TMC24x Stepper Motor Driver

In this chapter specific information pertaining to the setup of TMC23x and TMC24x is provided.

TMC23x/24x Support

TMC4361 provides the following features in order to support the TMC23x motor stepper driver family well:

- Automatic Mixed Decay chopper mode
- ChopSync
- Automatic switchover between microstep and fullstep operation
- Controlled PWM signal generation and automatic switchover between SPI and PWM mode; see section [13.2](#) (page [133](#)).

In the following section, the features are explained in greater detail.

- i For further information, please refer to the manual of the particular stepper driver motor.

10.5.1. TMC23x Setup

In order to activate the SPI data transfer and SPI feature set for a connected TMC23x stepper motor driver, do as follows:

Action:

- Set `spi_output_format` = b'1000 (`SPI_OUT_CONF` register 0x04).
- Set `COVER_DATA_LENGTH` = 0 (`SPI_OUT_CONF` register 0x04).

Result:

TMC23x is selected as connected stepper motor driver.

10.5.2. TMC24x Setup

In order to activate the SPI data transfer and feature set for a connected TMC24x stepper motor driver, do as follows:

Action:

- Set `spi_output_format` = b'1001 (`SPI_OUT_CONF` register 0x04).
- Set `COVER_DATA_LENGTH` = 0 (`SPI_OUT_CONF` register 0x04).

Result:

TMC24x is selected as connected stepper motor driver.

- i In addition to the TMC23x features mentioned above, the TMC24x stepper driver family provides three stallGuard bits as load measurement indicator. Therefore, the TMC24x stepper family is supported by the TMC4361 for the following:
 - Stall detection and
 - Stop-on-Stall behavior

•→ Turn to next page for more information.



10.5.3. TMC23x/24x Status Bits

TMC4361 maps the following status bits of TMC23x/24x stepper drivers – which are transferred with each SPI datagram – to the STATUS register 0x0F:

Status Register Mapping for TMC23x/24x		
STATUS bit @TMC4361	Status flag @TMC23x/24x	Description
STATUS (24)	UV	Undervoltage flag.
STATUS (25)	OT	Over temperature flag.
STATUS (26)	OTPW	Temperature prewarning flag.
STATUS (27)	OCA	Overcurrent flag for bridge A.
STATUS (28)	OCB	Overcurrent flag for bridge B.
STATUS (29)	OLA	Open load flag for bridge A.
STATUS (30)	OLB	Open load flag for bridge B.
STATUS (31)	OCHS	Overcurrent high side flag.

Table 43: Mapping of TMC23x/24x Status Flags

TMC23x/24x Microsteps

TMC4361 only forward new current data (*CURRENTA_SPI* and *CURRENTB_SPI* at register 0x7B) for TMC23x/TMC24x in case the upper five bits of one of the two 9-bit current values changes; because TMC23x and TMC24x current data consist of four bit current values and one polarity bit for each coil.

Consequently, alterations of the internal microstep resolution only apply in case the new microstep resolution is lower than 16 bits.

10.5.4. Automatic Fullstep Switchover for TMC23x/24x

Because SPI current data is transmitted, automatic switchover from microsteps to fullsteps and vice versa is only dependent on the internal ramp velocity.

In order to activate automatic switchover between microstep and fullstep operation, do as follows:

Action:

- Set *FS_VEL* register 0x60 according to the velocity [pps] at which the switchover must happen.
- Set *fs_en* = 1 (bit19 of *GENERAL_CONF* register 0x00).

Result:

Now, current values are switched to fullstep values in case $|V_{ACTUAL}| \geq FS_VEL$.
A switchback from fullsteps to μ steps is executed in case $|V_{ACTUAL}| < FS_VEL$.
The status bit *FS_ACTIVE* is set active as long as fullstep mode is enabled and activated.

•→ Turn to next page for more information.



10.5.5. Mixed Decay Configuration for TMC23x/24x

TMC4361 supports the mixed decay feature for the TMC23x/24x chopper in `SPI_OUT_CONF` register 0x04.

In order to configure mixed decay bits for TMC23x/24x, do as follows:

Action:

- Set `mixed_decay` = b'00 if mixed decay must always be deactivated.
- Set `mixed_decay` = b'01 if mixed decay must be activated for each coil during the falling ramp of the sine curve until reaching value 0.
- Set `mixed_decay` = b'10 if mixed decay must always be activated, except during standstill.
- Set `mixed_decay` = b'11 if mixed decay must always be activated.

Result:

The mixed decay bits for TMC23x/24x stepper motor drivers are set according to the configuration and the internal MSLUT values.

- i Please refer to the TMC23x/TMC24x datasheets to get more information about the configuration of mixed decay bits.

10.5.6. ChopSync Configuration for TMC23x/24x Stepper Drivers

TMC4361 forwards the internal clock at the output pin `STDBY_CLK`. This pin can also be used to provide an external clock for the TMC23x/24x stepper motor driver. This external clock generator automatically generates clock cycles that are modified by the chopSync feature if TMC23x/24x is configured as connected motor driver. Using chopSync enhances the motor drive for fast and smooth operation.

In order to enable the chopSync clock via the `STDBY_CLK` pin, do as follows:

Action:

- Set `CHOPSYNC_DIV` register 0x1F to generate an external clock frequency f_{OSC} according to the following equation: $f_{OSC} = f_{CLK} / CHOP_SYNC_DIV$.
- Set `stdby_clk_pin_assignment` = b'10 (`GENERAL_CONF` register 0x00).

Result:

`STDBY_CLK` generates an external clock with the selected frequency f_{OSC} that automatically provides the chopSync feature.

- i Recommended minimum external frequency f_{OSC} : two times higher than audible range.

Connection of `STDBY_CLK` output pin of TMC4361 and `OSC` input pin of TMC23x/24x¹

NOTICE

Risk of Burns! Avoid overheating and damage of the TMC23x/24x stepper driver and damage of the connected motor!

- You MUST use a low pass filter between `STDBY_CLK` output of TMC4361 and the `OSC` input pin of TMC23x/24x.
- You MUST keep the external clock frequency of the TMC23x/24x stepper motor driver below 50 kHz (to prevent overheating).

This will ensure smooth and safe operation.

¹ Per default (i.e. after power on and reset), `STDBY_CLK` forwards the internal clock that is too high for the TMC23x/24x. See Figure 10, (page 15) that provides a properly connected sample hardware setup.



10.5.7. Using TMC24x stallGuard Characteristics

TMC24x forwards stallGuard values $=\{LD2\&LD1\&LD0\}$ instead of one stallGuard2 status bit. These bits represent an unsigned value between 0 and 7. The lower the value is the higher the mechanical load is. TMC4361 can generate a one-bit internal stall signal by analyzing the stallGuard values.

In order to set up the stall load limit for automatic stall recognition, do as follows:

Action:

- Set proper *STALL_LOAD_LIMIT* (bit10:8 of *SPIOUT_CONF* register 0x04).

Result:

Whenever $\{LD2\&LD1\&LD0\} \leq \text{STALL_LOAD_LIMIT}$ a stall is indicated.

This feature also allows use of the Stop-on-Stall feature – already explained in section [10.4.4](#), page [104](#) – because this also applies to other TMC motor stepper drivers.

Additionally, a standby datagram can be sent automatically when a Stop-on-Stall is executed. In order to activate this behavior, do as follows:

Action:

- Set *VSTALL_LIMIT* register 0x67 [pps] according to minimum absolute velocity value for a correct stall recognition.
- Set *stop_on_stall* = 1 (bit26 of *REFERENCE_CONF* register 0x01).
- Set *drive_after_stall* = 0 (bit27 of *REFERENCE_CONF* register 0x01).
- Set *stdby_on_stall_for_24x* = 1 (bit6 of *SPIOUT_CONF* register 0x04).

Result:

Whenever a stall is calculated by comparing *STALL_LOAD_LIMIT* to the response of TMC24x, while at the same time the absolute value of *VACTUAL* exceeds *VSTALL_LIMIT*, the internal ramp velocity is stopped immediately. Additionally, both current values are then set to 0 whereupon a standby mode for the TMC24x stepper motor driver is generated that switches off all power driver outputs and clears the error flags.

- i To return from Stop-on-Stall, *drive_after_stall* must be set manually, as stated further in section [10.4.4](#) (Page [104](#)).

In order to exchange the UV status bit in the *STATUS* register 0x0F with the calculated stallGuard bit, do as follows:

Action:

- Set *stall_flag_instead_of_uv_en* = 1 (bit10:8 of *SPIOUT_CONF* register 0x04).

Result:

STATUS (24) shows the calculated stallGuard bit by comparing *STALL_LOAD_LIMIT* with the received response datagram of TMC24x.



10.6. TMC26x Stepper Motor Driver

TMC26x Stepper Motor Driver Support

TMC4361 provides the following features in order to support the TMC26x motor stepper driver family well:

- SPI mode that sets up current values directly.
- S/D mode in which the TMC26x processes S/D outputs of TMC4361.
- Automatic switchover between microstep and fullstep operation for both modes.
- Stall detection and Stop-on-Stall behavior for both modes.
- S/D mode only: Transfer of automatic scaling values from TMC4361 to TMC26x.
- S/D mode only: Transfer of auto-generated polling datagrams sent by TMC4361 for reception of status data and microstep position from TMC26x.

In the following section, the features are explained in greater detail.

- i For more information, please refer to the manual of the connected stepper driver motor.

10.6.1. TMC26x Setup (SPI mode)

In order to activate the SPI data transfer mode and feature set for a connected TMC26x stepper motor driver, do as follows:

Action:

- Set `spi_output_format` = b'1010 (`SPI_OUT_CONF` register 0x04).
- Set `COVER_DATA_LENGTH` = 0 (`SPI_OUT_CONF` register 0x04).

Result:

TMC26x in SPI mode is selected as connected stepper motor driver. Cover datagrams and current datagrams are sent via SPI output pins.

10.6.2. TMC26x Setup (S/D mode)

In order to activate the S/D mode and feature set for a connected TMC26x stepper motor driver, do as follows:

Action:

- Connect SPI output pins and S/D outputs to the TMC26x stepper motor driver.
- Set `spi_output_format` = b'1011 (`SPI_OUT_CONF` register 0x04).
- Set `COVER_DATA_LENGTH` = 0 (`SPI_OUT_CONF` register 0x04).
- Set `DIR_SETUP_TIME` and `STP_LENGTH_ADD` (register 0x10) according to the hardware setup.
- Set proper `POLL_BLOCK_MULT` (bit12:7 of `SPIOUT_CONF` register 0x04).

Result:

TMC26x in S/D mode is selected as connected stepper motor driver. SPI output pins transfer only cover datagram and automatic configuration datagrams because motion is generated by processing the STPOUT/DIROUT output signals of TMC4361. The next polling datagram is sent (`POLL_BLOCK_MULT+1`)*`SPI_BLOCK_TIME` clock cycles after the last polling datagram.

- i A high microstep frequency requires a short SPI datagram polling time.

•→ *Continued on next page.*



10.6.3. Sending Cover Datagrams to the TMC26x Driver

Based on the TMC26x settings - that were explained above - TMC4361 now sends 20-bit datagrams automatically.

In order to send cover datagrams to TMC26x motor stepper drivers, do as follows:

Action:

- Set *COVER_LOW* (19:0) to the register values that need to be transferred.

Result:

A cover datagram is sent to the connected driver. *COVER_DONE* is set after data transfer. The response of TMC26x is stored in *COVER_DRV_LOW* (19:0).

10.6.4. TMC26x SPI Mode: Automatic Fullstep Switchover

Because SPI current data is transmitted, the automatic switchover from microsteps to fullsteps and vice versa entirely depends on the internal ramp velocity.

In order to activate automatic switchover between microstep and fullstep operation, do as follows:

Action:

- Set *FS_VEL* register 0x60 according to the absolute velocity [pps] at which the switchover should happen.
- Set *fs_en* = 1 (bit19 of *GENERAL_CONF* register 0x00).

Result:

Now, current values are switched to fullstep values, in case $|V_{ACTUAL}| \geq FS_VEL$.
A switchback from fullsteps to μ steps is executed in case $|V_{ACTUAL}| < FS_VEL$.

The status bit *FS_ACTIVE* is set active as long as fullstep mode is enabled and activated.

•→ Continue on the next page for information on fullstep switchover for TMC26x in S/D mode.



10.6.5. TMC26x S/D Mode: Automatic Fullstep Switchover

In S/D mode, switchover from microsteps to fullsteps and vice versa is not only dependent on the internal ramp velocity but also on the microstep position of the TMC26x MSLUT; because switching to a lower resolution must be executed carefully to catch the correct microstep position. Proper setting of read selection bits for TMC26x stepper drivers TMC4361 is required to execute switchover automatically.

In order to activate automatic switchover between microstep and fullstep operation in TMC26x S/D mode, do as follows:

PRECONDITION:

Mandatory TMC26x configuration MUST be executed via cover datagrams:

- Set RDSEL1 = 0 and RDSEL0 = 0 @TMC26x.

Action:

- Set *disable_polling* = 0 (bit6 of *SPI_OUT_CONF* register 0x04).
- Set *FS_VEL* register 0x60 according to the absolute switching velocity [pps].
- Set *fs_en* = 1 (bit19 of *GENERAL_CONF* register 0x00).
- Set *fs_sdout* = 0 (bit20 of *GENERAL_CONF* register 0x00).

Result:

The μ step resolution of TMC26x is set to fullsteps, in case $|V_{ACTUAL}| \geq FS_VEL$. A switchback from fullsteps to μ steps is executed in case $|V_{ACTUAL}| < FS_VEL$. *FS_ACTIVE* is set active as long as fullstep mode is enabled and activated. Presettings of the TMC26x DRVCTRL register – *that is executed beforehand via cover datagrams* – are considered whenever the particular register is overwritten with a newly assigned microstep resolution.

10.6.6. TMC 26x S/D Mode: Change of Current Scaling Parameter

SPI mode-supported TMC26x drivers are automatically scaled by means of current datagrams. In order to automatically scale the current of a connected TMC26x motor stepper driver in S/D mode, TMC4361-LA sends auto-generated cover datagrams by altering directly the CS value of the TMC26x SGCSCONF register.

TMC4361 provides features that change the current scaling automatically, which are explained in chapter 11, page 120.

In order to activate automatic current scaling for a connected TMC26x in S/D mode, do as follows:

Action:

- Set *scale_val_transfer_en* = 1 (bit5 of *SPI_OUT_CONF* register 0x04).
- Set the scale value register 0x06 and scale configuration register 0x05 according to your requirements (see chapter 11, page 120).

Result:

If the current scaling is adapted internally, TMC4361 automatically sends cover datagrams to TMC26x that change the CS bit directly. Presettings of the TMC26x SGCSCONF register – *that are executed beforehand via cover datagrams* – become considered whenever the particular register is overwritten with a newly assigned current scaling value.

NOTE:

- Please consider that the CS value consists of 5 bits only. Therefore, the scaling values in register 0x06 must be adapted to 5-bit values as well.



10.6.7. TMC26x Status Bits

TMC4361 maps the following status bits of TMC26x stepper drivers – which are transferred within each SPI response – to the *STATUS* register 0x0F:

Status Register Mapping for TMC26x		
STATUS Bit @TMC4361	Status Flag @TMC26x	Description
STATUS(24)	SG	stallGuard2™ status flag
STATUS(25)	OT	Over temperature flag
STATUS(26)	OTPW	Temperature prewarning flag
STATUS(27)	S2GA	Short-to-ground detection flag for high side MOSFET of coil A
STATUS(28)	S2GB	Short-to-ground detection flag for high side MOSFET of coil B
STATUS(29)	OLA	Open load flag for bridge A
STATUS(30)	OLB	Open load flag for bridge B
STATUS(31)	STST	Standstill flag

Table 44: Mapping of TMC26x Status Flags

- i If polling is not disabled (*disable_polling* = 0), status data from TMC26x is also available in S/D mode.

10.7. TMC389 Stepper Motor Driver

Configuration for the TMC389 3-Phase Stepper Driver

If a TMC389 is connected to the SPI output and a microstep resolution of 256 is set, a 3-phase stepper output for coil B can be generated. All features of TMC26x stepper motor drivers in SPI mode are also available for TMC389.

In order to activate the SPI data transfer mode and feature set - for a connected TMC389 3-phase stepper motor driver - do as follows:

Action:

- Set *spi_output_format* = b'1010 (*SPI_OUT_CONF* register 0x04).
- Set *three_phase_stepper_en* = 1 (*SPI_OUT_CONF* register 0x04).
- Set *COVER_DATA_LENGTH* = 0 (*SPI_OUT_CONF* register 0x04).

Result:

Now, the *CURRENTB* and *CURRENTB_SPI* values are shifted by 120° towards *CURRENTA* and *CURRENTA_SPI* — in contrast to the 90° shift of the 2-phase stepper motors.



10.8. TMC2130 Stepper Motor Driver

TMC2130 Support

TMC4361 provides the following features in order to support the TMC2130 motor stepper driver well:

- SPI mode that sets up current values directly.
- S/D mode in which the TMC2130 processes S/D outputs of TMC4361.
- Automatic switchover between microstep and fullstep operation for both modes.
- Stall detection and Stop-on-Stall behavior for both modes.
- S/D mode only: Transfer of automatic scaling datagrams from TMC4361 to TMC2130.
- S/D mode only: Transfer of auto-generated polling datagrams sent by TMC4361 for reception of status data and microstep position from TMC2130.

In the following section, the features are explained in greater detail.

- i For more information, please refer to the manual of the TMC2130 stepper driver motor.

10.8.1. Set-up TMC2130 Support (SPI Mode)

In order to activate the SPI data transfer mode and feature set - for a connected TMC2130 stepper motor driver - do as follows:

Action:

- Set *spi_output_format* = b'1101 (*SPI_OUT_CONF* register 0x04).
- Set *COVER_DATA_LENGTH* = 0 (*SPI_OUT_CONF* register 0x04).

Result:

TMC2130 in SPI mode is selected as connected stepper motor driver. Cover datagrams and current datagrams are sent via SPI output pins.

10.8.2. Set-up TMC2130 Support (S/D Mode)

In order to activate the S/D mode and feature set - for a connected TMC2130 stepper motor driver - do as follows:

Action:

- Connect SPI output pins and S/D outputs to the TMC2130 stepper motor driver.
- Set *spi_output_format* = b'1100 (*SPI_OUT_CONF* register 0x04).
- Set *COVER_DATA_LENGTH* = 0 (*SPI_OUT_CONF* register 0x04).
- Set *DIR_SETUP_TIME* and *STP_LENGTH_ADD* (register 0x10) according to the hardware setup.
- Set proper *POLL_BLOCK_MULT* (bit12:7 of *SPIOUT_CONF* register 0x04).

Result:

TMC2130 in S/D mode is selected as connected stepper motor driver. SPI output pins transfer only cover datagrams and automatic configuration datagrams because motion is generated by processing the STPOUT/DIROUT output signals of TMC4361. The next polling datagram is sent (*POLL_BLOCK_MULT*+1) · *SPI_BLOCK_TIME* clock cycles after the last polling datagram.

- i A high microstep frequency requires a short SPI datagram polling time.



10.8.3. Sending Cover Datagrams to the TMC2130 Driver

Based upon the TMC2130-supported settings explained above, the TMC4361 now sends 40 bit datagrams automatically.

In order to send cover datagrams to TMC2130 motor stepper drivers, do as follows:

Action:

- Set *COVER_HIGH* (7:0) register 0x6D to the address value that need to be transferred.
- Set *COVER_LOW* (31:0) register 0x6C to the data values that need to be transferred.

Result:

A cover datagram is sent to the connected driver. *COVER_DONE* is set after data transfer. The response of TMC2130 is stored in *COVER_DRV_HIGH* (7:0) and *COVER_DRV_LOW* (31:0)

10.8.4. TMC2130 SPI Mode: Automatic Fullstep Switchover

Because SPI current data is transmitted, the automatic switchover from microsteps to fullsteps and vice versa entirely depends on the internal ramp velocity.

In order to activate automatic switchover between microstep and fullstep operation, do as follows:

Action:

- Set *FS_VEL* register 0x60 according to absolute velocity [pps] at which the switchover should happen.
- Set *fs_en* = 1 (bit19 of *GENERAL_CONF* register 0x00).

Result:

Now, current values are switched to fullstep values, in case $|V_{ACTUAL}| \geq FS_VEL$. A switchback from fullsteps to μ steps is executed in case $|V_{ACTUAL}| < FS_VEL$. The status bit *FS_ACTIVE* is set active as long as fullstep mode is enabled and activated.

10.8.5. TMC2130 S/D Mode: Automatic Fullstep Switchover

During S/D mode, switchover from microsteps to fullsteps and vice versa is only executed directly by TMC2130. Therefore, a fullstep velocity must only be defined in TMC2130. TMC4361 transfers microsteps whether TMC2130 is operating in fullstep or microstep mode.



10.8.6. TMC 2130 S/D Mode: Changing current Scaling Parameter

TMC4361 provides features that change the current scaling automatically, which is explained in Chapter 11, page 120. Stepper motor drivers that are supported by SPI current datagrams are automatically scaled via current datagrams. To automatically scale the current of a connected TMC2130 motor stepper driver in S/D mode, TMC4361-LA sends auto-generated cover datagrams by altering the CS value of the TMC2130 IHOLD_IRUN register.

In order to activate automatic current scaling for TMC2130 in S/D mode:

Action:

- Set *scale_val_transfer_en* = 1 (bit5 of *SPI_OUT_CONF* register 0x04).
- Set scale value register 0x06 and scale configuration register 0x05 according to your requirements (see chapter 11, page 120).

Result:

When current scaling is adapted internally, TMC4361 sends cover datagrams to TMC2130 automatically, which changes the CS bit directly.

Presettings of the IHOLD_IRUN register of the TMC2130 – executed before via cover datagrams – are considered whenever the particular register is overwritten with a newly assigned current scaling value.

- i Please consider that the IRUN and IHOLD values consist of 5 bits only. Therefore, scaling values in register 0x06 must also be adapted to 5-bit values.

TMC2130 Status Bits

TMC4361 maps the following status bits of TMC2130 stepper drivers – *which are transferred within each SPI response* – to the *STATUS* register 0x0F:

Status Register Mapping for TMC2130		
STATUS Bit @TMC4361	Status Flag @TMC2130	Description
STATUS (24)	SG	stallGuard2™ status flag.
STATUS (25)	OT	Over temperature flag.
STATUS (26)	OTPW	Temperature prewarning flag.
STATUS (27)	S2GA	Short-to-ground detection flag for high side MOSFET of coil A.
STATUS (28)	S2GB	Short-to-ground detection flag for high side MOSFET of coil B.
STATUS (29)	OLA	Open load flag for bridge A.
STATUS (30)	OLB	Open load flag for bridge B.
STATUS (31)	STST	Standstill flag.

Table 45: Mapping of TMC2130 Status Flags

- i If polling is not disabled (*disable_polling* = 0), status data from TMC2130 is also available in S/D mode.



10.9. Connecting Non-TMC Stepper Motor Driver or SPI-DAC at SPI output interface

TMC4361 also provides configuration data for driver chips of other companies via the cover registers. The following output format settings can be selected:

Non-TMC Data Transfer Options		
Output Formats	<i>spi_output_format</i>	Comment
SPI output off	b'0000	SPI output driver pins are switched off.
Cover output only	b'1111	Only cover datagrams are sent via the SPI output pins.
Unsigned scaling factor	b'0100	The actual unsigned current scaling value is provided at the SPI output pins.
Signed current data	b'0101	Both actual signed current values are provided in one datagram at the SPI output pins.
DAC scaling factor	b'0110	The actual unsigned current scaling value is provided at the SPI output pins for a defined DAC address.
DAC absolute values	b'0011	Both actual signed current values are provided in two datagrams at the SPI output pins for defined DAC addresses, which are absolute values. Phase bits are generated at the STPOUT/DIROUT interface. Phase bit = 0 signifies positive values.
DAC absolute values	b'0010	Both actual signed current values are provided in two datagrams at the SPI output pins for defined DAC addresses, which are absolute values. Phase bits are generated at the STPOUT/DIROUT interface. Phase bit = 1 signifies positive values.
DAC adapted values	b'0001	Both actual signed current values are provided in two datagrams at the SPI output pins for defined DAC addresses. These values are mapped to positive values: Current value equals minimum value (-255) = 0 Current value equals 0 = 128 Current value equals maximum value (+255) = 255

Table 46: Non-TMC Data Transfer Options

NOTE:

→ Please note that the *COVER_DATA_LENGTH* must be set according to the predefined driver chip datagram length.

Cover Output only



In order to send cover datagrams only, use this option to avoid datagrams that send scaling or current values whenever these internal values are changed.

Please keep in mind that only the SPI protocol is available that is used for TMC motor stepper drivers.

Sending unsigned Scaling Factor

Setting *spi_output_format* = b'0100 leads to a transfer of the 8-bit scaling factor if this value is altered internally: Output data(7:0) = *SCALE_PARAM* (7:0).

The MSB 7 is sent first. If more than 8 bits are configured as *COVER_DATA_LENGTH*, leading zeros are inserted before the MSB.

Sending signed Current Values

Setting *spi_output_format* = b'0101 leads to a transfer of both signed current values that consists of 18 bits and are sent one after the other in one datagram: Output data(17:0) = *CURRENTA_SPI* (8:0) & *CURRENTB_SPI* (8:0).

The MSB (bit17) is sent first. If more than 18 bits are configured as *COVER_DATA_LENGTH*, leading zeros are inserted before the MSB.



10.9.1. Connecting a SPI-DAC

DAC Output Values

Connecting a compatible SPI-DAC to SPI output pins, several possibilities are available for output configuration:

- Output of the internal SPI current values.
- Output of the internal current scaling value.
- Several SPI protocols are available.

10.9.2. DAC Data Transfer

SPI-DACs can convert more than one digital value, but every value is transmitted in one datagram. Because TMC4361 provides two current values, a datagram transfer from TMC4361 to a connected SPI-DAC is split into two datagrams, one for each current value: *CURRENTA_SPI* and *CURRENTB_SPI*.

The transmission is initiated as soon as one of both values is changed internally. The data transfer of the second current value *CURRENTB_SPI* is executed automatically whenever the transmission of *CURRENTA_SPI* is completed.

If only the scaling factor *SCALE_PARAM* needs to be transferred, only one datagram is sent out.

10.9.3. Changing SPI Output Protocol for SPI-DAC

Per default, the SPI protocol follows the TMC style: To initiate a data transfer, the negated chip select signal NSCSDRV_SDO switches from high to low level. After a while, the serial clock SCKDRV_NSDO switches from high to low level. When the transmission is finished, the serial clock switches to high level. Afterwards, the negated chip select signal switches to high level to finish the data transfer.

Adaptations to suit other SPI protocols are also available:

In order to set serial clock to low level - before the negated chip select switches to low level - do as follows:

Action:

- Set *sck_low_before_csn* = 1 (bit4 of *SPIOUT_CONF* register 0x04).

Result:

SCKDRV_NSDO is tied low before NSCSDRV_SDO switches to low level to initiate data transfer.

Per default, TMC drivers sample master data with the rising edge of the serial master clock. Thus, TMC4361 shifts output data at SDODRV_SCLK with the falling edge of SCKDRV_NSDO.

If the data must be sampled with the falling edge of the master clock at the driver's side, do as follows:

Action:

- Set *new_out_bit_at_rise* = 1 (bit5 of *SPIOUT_CONF* register 0x04).

Result:

The output data at SDODRV_SCLK is changed with the rising edge of SCKDRV_NSDO.



10.9.4. DAC Address Values

SPI transmission to a DAC transfers an address or a command prior to the value that must be defined. The length of the prefixed command/address can be assigned by setting *DAC_CMD_LENGTH* according to specification of the SPI-DAC.

In order to set up the DAC communication scheme, do as follows:

Action:

- Set *DAC_CMD_LENGTH* (bit11:7 of *SPI_OUT_CONF* register 0x04) according to the length of the address / command, which is placed in front of the values.
- Set *DAC_ADDR* register 0x1D according to your requirements:
Address/command of the 1st value: Set DAC_ADDR(15:0) = DAC_ADDR_A.
Address/command of the 2nd value: Set DAC_ADDR(31:16) = DAC_ADDR_B.

Result:

DAC_ADDR_A is placed in front of the first transferred value that can be the current value of coilA (=CURRENTA_SPI) or the scaling factor (=SCALE_PARAM), whereas *DAC_ADDR_B* is placed before the second current value *CURRENTB_SPI*.

- i *COVER_DATA_LENGTH* comprises the whole datagram length, which is the sum of the address/length *DAC_CMD_LENGTH* and the 8-bit data length.
- i If the cover register length comprises more bits than the combination of address/command and value, trailing zeros are added at the end.
- i The command bits consist of the least significant bits of *DAC_ADDR_x* if the command length is less than 16 bits long.

10.9.5. DAC Data Values

Several opportunities are available for the DAC data style:

- Current values are converted to absolute values. The phases of the values are generated at the STPOUT (coilA) and DIROUT (coilB) pins. The base line (value equals 0) is located at 0 (see [Table 47](#), **Figures B and C**).
- The current values, *which range between -255 and 255*, are mapped to values between 0 and +255: the minimum value of -255 is an output value of 0, whereas the baseline is set to +128. The maximum value remains at +255. In detail, the value is divided by two and 128 is added to the quotient ([Table 47](#), page [119](#), **Fig. A**).

TMC4381 provides an offset to compensate for a shifted DAC baseline.

In order to shift the DAC baseline, do as follows:

Action:

- Set *DAC_OFFSET* (bit31:24 of register 0x7E) according to your requirements.

Result:

The digital values are shifted accordingly. [Table 47](#) (page [119](#)), **Figure D** shows absolute DAC values. The DAC baseline is shifted by 32 steps, whereas [Table 47](#) (page [119](#)), **Figure E** shows mapped DAC values, which are shifted by 64 steps.

- i For the three available absolute values options – including the unsigned scale parameter transfer – the offset represents an unsigned number.
- i For the mapped values option the offset represents a signed number. To avoid a carry over at the value limits +255 and -256 when using an DAC offset, the MSLUT values must be scaled down for the SPI output values (see [Table 47](#) (page [119](#)), **Figures D and E**). This can be done by using the current scale feature, as explained in chapter [11](#), page [120](#).

- i •→ Continued on next page.



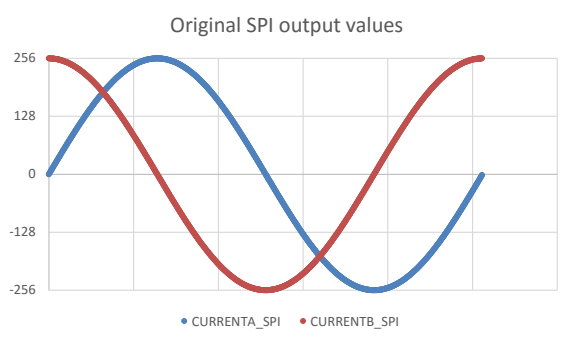
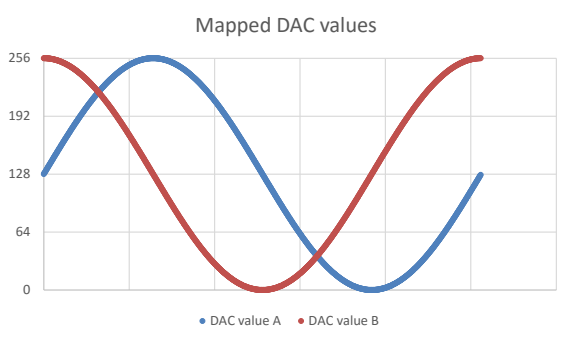
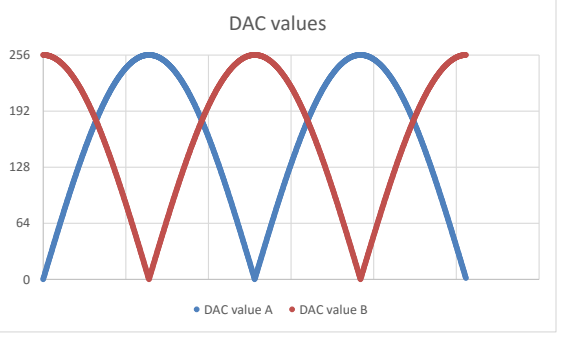
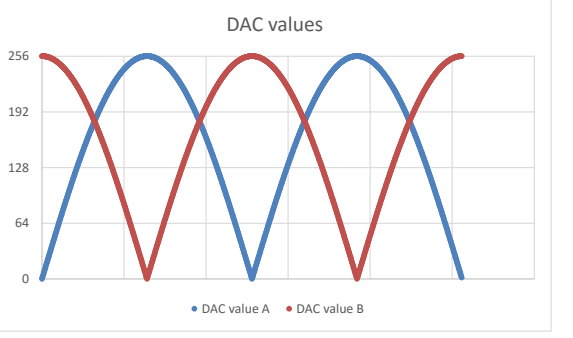
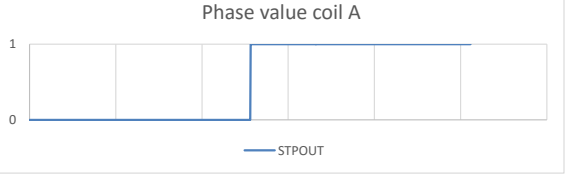
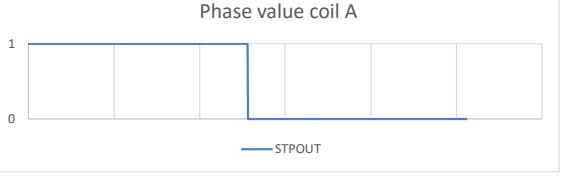
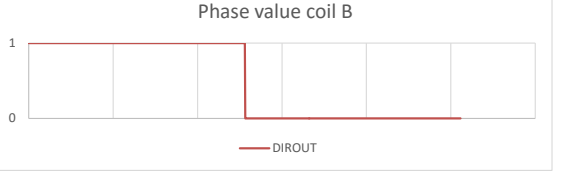
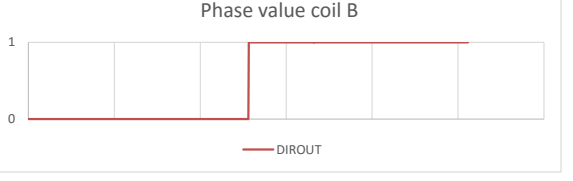
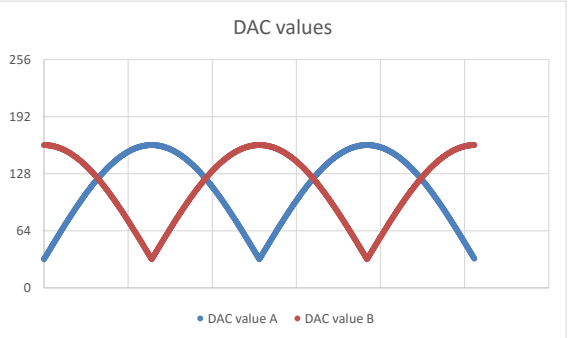
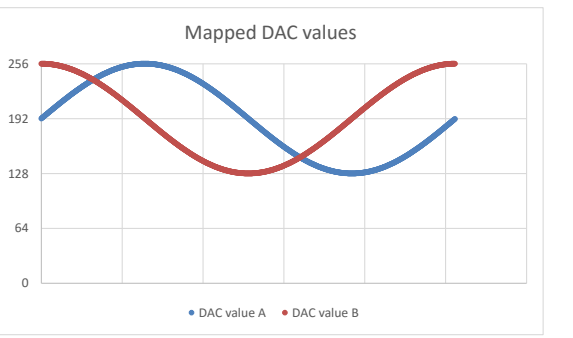
Available DAC Options for the SPI Output Interface	
Original SPI Output Curves:	A Mapped DAC values:
	
B Absolute DAC values (positive phase = 0)	C Absolute DAC values (positive phase = 1)
	
	
	
D Absolute DAC values, original MSLUT values are scaled to ½, DAC value offset=32	E Mapped DAC values, original MSLUT values are scaled to ½, DAC value offset = 64
	

Table 47: Available SPI-DAC Options



11. Current Scaling

The current values of register 0x7A – *CURRENTA* and *CURRENTB* – of the microstep lookup table (MSLUT) represent the maximum 9-bit signed values, which can be sent via the SPIOUT output interface. In most sections of the velocity ramp it is not required to drive the motor with the full current amplitude. Various possibilities are implemented that allow adaptation of actual current values of the MSLUT to the present ramp status. Scale parameters are available for boost current, hold current, and drive current.

These parameters can be assigned independently in the *SCALE_VALUES* register 0x06, and are used automatically for different states of the velocity ramp; if enabled, as described below. Prior to describing the various feasible scaling situations, a brief explanation of the scaling calculation is provided.

Calculation of the Current Output Values

Description of Scaling Calculation

When scaling is enabled for the present ramp state, the actual current values of the MSLUT are multiplied with the *MULT_SCALE* parameter that is deduced from one of the four *SCALE_VALUES*:

$$\begin{aligned} \text{MULT_SCALE} &= (\text{actual_SCALE_VAL} + 1) / 256 \\ \text{with actual_SCALE_VAL} &= \{\text{HOLD, BOOST, DRV1, DRV2}\}. \end{aligned}$$

Consequently, this *MULT_SCALE* ranges from 0 to 1: $0 < \text{MULT_SCALE} \leq 1$.

MULT_SCALE is then multiplied with the actual current values *CURRENTA* and *CURRENTB*, which are generated by the MSLUT:

$$\begin{aligned} \text{CURRENTA_SPI} &= \text{CURRENTA} \cdot \text{MULT_SCALE} && (\text{bit8:0 of 0x7B}) \\ \text{CURRENTB_SPI} &= \text{CURRENTB} \cdot \text{MULT_SCALE} && (\text{bit24:16 of 0x7B}) \end{aligned}$$

These values are transferred via SPI output interface. If no current scaling is enabled, the output values *CURRENTA_SPI* and *CURRENTB_SPI* are equal to the MSLUT values *CURRENTA* and *CURRENTB* because the scaling values are equal to the maximum 255, per default. Thus, scaling will only decrease the original MSLUT values.

Also, the actual scale parameter can assume intermediate values because TMC4361 offers possibilities to convert smoothly from one scale value to another. The actual scale parameter *SCALE_PARAM* can be read out at register 0x7C. It has the same range as the four *SCALE_VALUES*.

AREAS OF SPECIAL CONCERN

- Use of TMC26x and TMC2130 stepper motor drivers in S/D mode:**
 If TMC motor stepper drivers are used in S/D mode, scaling values comprise only 5 bits because the CS value of TMC26x, and the IHOLD, IRUN values of TMC2130 motor stepper drivers are adapted directly. Therefore, *MULT_SCALE* is calculated slightly differently:

$$\text{MULT_SCALE} = (\text{actual_SCALE_VAL} + 1) / 32$$

